



Athena: An AI-Powered Campus Query Bot

Project Proposal for UCS503

Team: The 3 Stooges

Submitted to: Ms. Jhonsy Bansal

Thapar Institute of Engineering and Technology

Armandeep Singh Taggar, CSED

Roll No: 1024030219

Sanchit Narula, CSED

Roll No: 1024031048

Nipun Gupta, CSED

Roll No: 1024031169

August 31, 2026

1 Higher-order goal

... secondary framing

This project aims to improve the student experience during and after college admission by reducing the time and effort required to find accurate campus information. While the underlying problem is broader than any single institution, the immediate engineering objective is to translate *centralized, verified access to campus knowledge* into a measurable, deployable software solution.

2 Time-to-value

... primary heuristic

- We will deliver meaningful increments quickly by prototyping the core query-bot workflow (ask a question, retrieve from a verified knowledge base, get a grounded answer) and validating it within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations to reduce release friction, starting with a single-college deployment before generalizing to multiple colleges.
- Success is defined by what can be delivered and measured in the first iteration (a working query bot on institutional data), then improved based on user feedback.

3 Problem Statement

Having experienced this ourselves, we identified a common challenge faced by students both before and after joining college: accessing accurate, timely, and centralized campus information. Common pain points include:

- **Fragmentation:** official notices, events, venues, society information, and academic details are scattered across multiple websites, portals, PDFs, and social media pages.
- **Low discoverability:** students cannot easily search or navigate this scattered information, especially newcomers unfamiliar with the campus.
- **No conversational access:** existing sources require manual searching rather than allowing a student to simply ask a question in natural language.
- **No verification layer:** there is no reliable way to confirm that information (or the person accessing it) genuinely belongs to a given institution.

Why this matters now (contextual relevance): As colleges publish more information digitally but across disconnected channels, even a modest centralization and search layer can noticeably reduce the time students spend hunting for basic information.

4 Proposed Solution

... Software Proposition

4.1 Overview

Build a **web-based**, multi-college platform, “Athena”, a centralized AI-powered campus assistant with the following components:

- **AI Campus Query Bot:** a natural-language interface for students to ask questions (e.g., “What is the hostel admission procedure?”) and receive answers grounded in verified institutional data.
- **Navigation:** an interactive 2D campus map (with a roadmap toward 3D) to help students locate classrooms, departments, facilities, and event venues.
- **Academics:** centralized course codes, faculty details, syllabi, timetables, date sheets, and a CGPA calculator.
- **Societies & Events:** consolidated information on student societies, upcoming events, and announcements.
- **Personalized Dashboard & Reminders:** a student-specific dashboard for managing reminders and important dates.
- **Admin access & student verification:** each institution maintains its own verified knowledge base, with a mechanism to confirm a student belongs to the selected institution.

4.2 Core workflow

1. Student selects/registers with their institution and is verified as belonging to it.
2. Student asks a question in natural language through the query bot.
3. The system identifies the student’s institution and retrieves relevant context from that institution’s verified knowledge base (semantic search over document embeddings).

4. The retrieved context is passed to the LLM, which generates a clear, contextual response grounded in the retrieved data.
5. For location-based queries, the response is additionally resolved into structured data (building, floor, coordinates) and visually surfaced on the campus map.

4.3 Operational constraints for an educational, multi-college setting

- Keep the solution usable on low-to-mid range devices via responsive web design.
- Ground every answer in retrieved, verified institutional data rather than relying on a general-purpose LLM alone, since general LLMs have no access to a college’s latest notices, room allocations, or internal rules.
- Design each institution’s knowledge base and admin access independently, so onboarding a new college does not require changing the underlying platform.

5 Solution Approach

... Engineering focus

This is an engineering course project, so we focus on system-level integration of existing, mature components rather than novel algorithm research.

5.1 Retrieval-grounded answering

... non-research

Answer generation is retrieval-augmented rather than a research contribution in itself:

- Documents (notices, syllabi, policies, etc.) are converted into vector embeddings using Sentence Transformers and stored via pgvector.
- A student query is embedded and matched against this store using semantic similarity, without claiming state-of-the-art retrieval.
- The retrieved context, not the raw query alone, is passed to the LLM (GPT-4o-mini) so answers stay grounded and easy to update — when college data changes, only the knowledge base needs updating, not the model.

5.2 Web architecture

... web-based solution

A typical multi-tier web architecture:

- **Frontend:** React.js (Vite) with Tailwind CSS for the student/admin interface, dashboards, and 2D map (with Three.js planned for interactive 3D navigation).
- **Backend API:** Django + DRF for authentication, admin management, and the API layer connecting the frontend to the RAG pipeline.
- **Database:** PostgreSQL with the pgvector extension to store students, colleges, courses, faculty, events, notices, and embeddings.
- **Document processing:** PyMuPDF to extract text from PDFs and college documents feeding the knowledge base.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run tests on every change (Git + GitHub).
- Produce repeatable deployment artifacts to staging (Render/Railway for backend, Vercel for frontend).
- Enable safe and frequent releases of incremental features as the knowledge base and feature set grow.

6 Evaluation Criterion

...measurable and attributable

We define success using measurable metrics tied to the core query-bot and navigation workflow.

6.1 Primary evaluation metric

Answer groundedness & relevance: the proportion of query-bot responses that are factually grounded in the retrieved institutional context and judged relevant by the asking student.

- **Target:** high grounded-answer rate on a curated RAG evaluation test dataset during the pilot window.
- **Attribution:** measured via RAG evaluation methodology comparing generated answers against the source documents retrieved.

6.2 Secondary evaluation metrics

- **Query response time:** end-to-end latency from question submission to answer display.
- **Navigation accuracy:** whether the correct building/floor/room is displayed/highlighted for location queries.
- **Student verification success rate:** percentage of legitimate students correctly verified against their institution.
- **Reliability:** availability of staging/production for login and querying during business hours.

6.3 Pilot validation plan

...high-level

- Recruit a pilot group of students (and an admin/staff role for knowledge-base maintenance) from one institution to begin with.
- Compare metrics before/after within the iteration window by logging query outcomes and gathering direct student feedback on answer usefulness.

7 Scalability

...with foresight

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in number of institutions, documents, and concurrent users by:
 - decoupling components (frontend/backend/RAG pipeline),

- indexing embeddings via pgvector for efficient semantic search at scale,
 - using a stateless API design so institutions can be onboarded without platform changes.
2. **Operationally deployable (practically):** The system should run on common web hosting:
- managed PostgreSQL database with pgvector,
 - platform-hosted deployment (Render/Railway, Vercel),
 - CI/CD pipeline for staging/production.

8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project:

- Web framework for REST APIs (Django + DRF) and a reactive frontend (React.js/Vite).
- Managed relational database with vector support (PostgreSQL + pgvector).
- Embedding model (Sentence Transformers) and LLM (OpenAI GPT-4o-mini) for the RAG pipeline.
- Document-processing library (PyMuPDF) for ingesting PDFs into the knowledge base.
- Authentication approach (Django Auth) and institutional-email-based student verification.
- Test framework (PyTest) and API testing tool (Postman) for unit/integration tests.
- CI runner and staging deployment targets (Render/Railway, Vercel).

We will use these mature components to focus on workflow design, grounded-answer correctness, and measurement rather than inventing new infrastructure.

8.1 Tech Stack / Tools Required

v1: tech we are using currently

Layer	Technology	Purpose	Status
Frontend	React.js (Vite)	Build the student/admin interface and dashboards	v1
Styling	Tailwind CSS	UI structure and styling	v1
Campus Navigation	2D map (SVG, floor-plan images)	Locate classrooms, departments, event venues	v1
Campus Navigation	Three.js	Interactive 3D campus navigation	roadmap
Backend	Django + DRF	Core logic, auth, admin management, API layer	v1
Database	PostgreSQL	Store students, colleges, courses, faculty, events, notices	v1
Vector search	pgvector	Store embeddings, semantic similarity search for RAG	v1

Embeddings	Sentence Transformers	Convert documents and queries into vector embeddings	v1
RAG pipeline	Custom Python pipeline	Retrieve documents, construct context for the LLM	v1
LLM	OpenAI GPT-4o-mini	Generate natural-language answers from retrieved context	v1
Document processing	PyMuPDF	Extract text from PDFs and college documents	v1
Authentication	Django Auth	Secure student and admin authentication	v1
Student verification	Institutional email check	Verify student belongs to selected institution	v1
Admin management	Django Admin	Maintain and update institutional information	v1
Location data	Structured JSON	Building, floor, room mapping for navigation	v1
API testing	Postman	Test backend APIs during development	v1
Version control	Git + GitHub	Code management and team collaboration	v1
Deployment	Render/Railway, Vercel	Host and serve backend, database, and frontend	v1

Roadmap: for future development — see Section 10 for the future-scope items planned beyond v1.

9 System Design

The figures below summarize the intended system design.

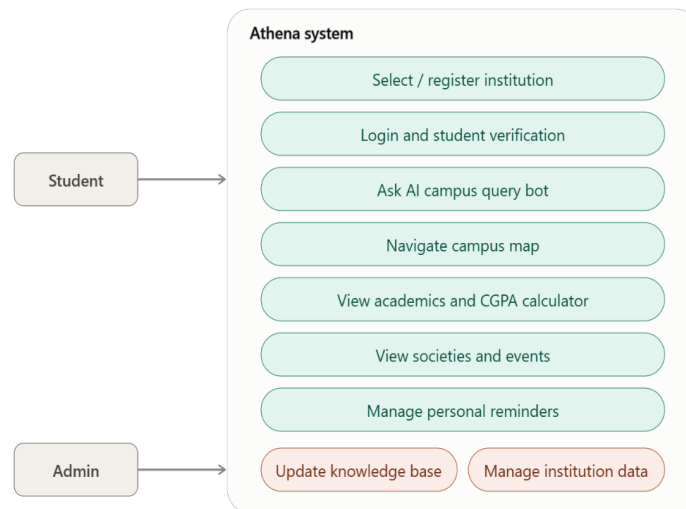


Figure 1: Case Diagram — students interact with the query bot, navigation, academics, and reminder features, while admins manage the institutional knowledge base.

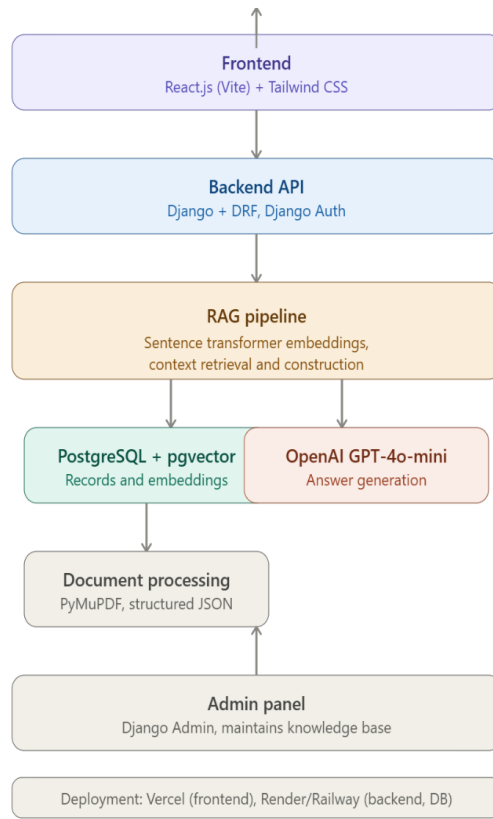


Figure 2: System Architecture — a student request travels from the React frontend down to the LLM, with the RAG pipeline in the middle grounding answers in verified data.

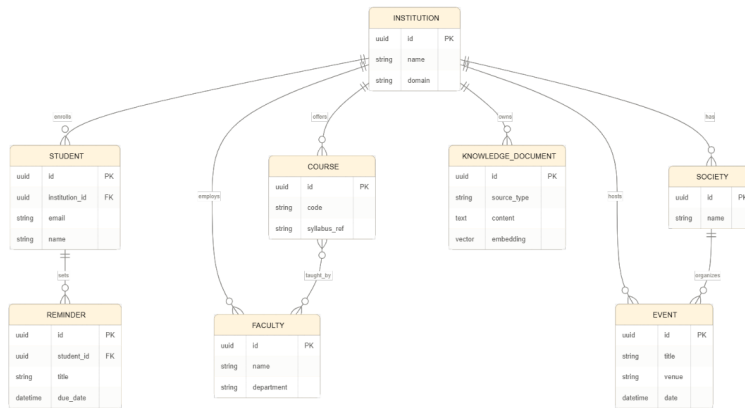


Figure 3: ER Diagram — core entities: Institution, Student, Course, Faculty, Event, Society, Reminder, and Knowledge_Document.

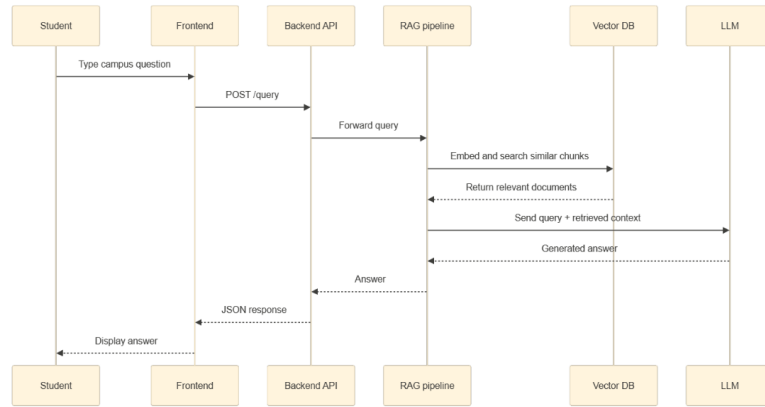


Figure 4: Sequence Diagram — message exchange for a single query across student, frontend, backend, RAG pipeline/vector store, and LLM.

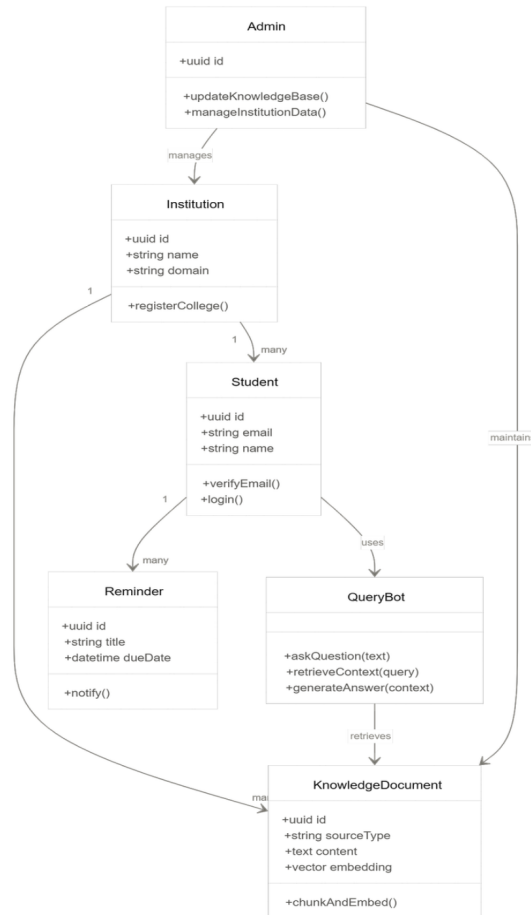


Figure 5: Class Diagram — core classes and their relationships, mirroring the ER diagram with added behavior relevant to the query bot and reminders.

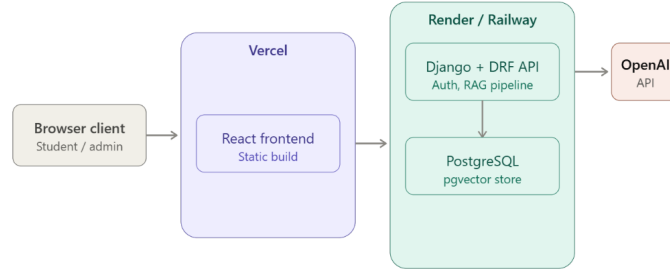


Figure 6: Deployment Diagram — browser client, Vercel frontend, Render/Railway backend and database, and the external OpenAI API.

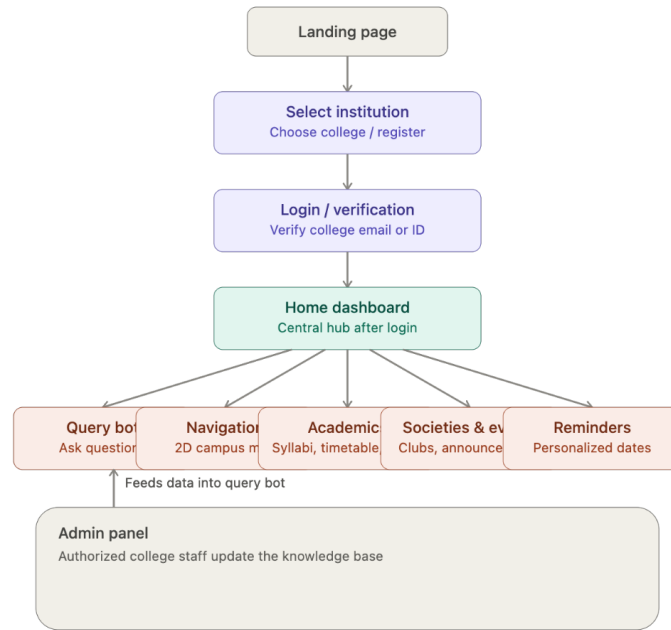


Figure 7: Athena Work Flow.

10 Project scope and deliverables

... *iteration-friendly*

10.1 Initial deliverable

... *first iteration*

- Student/admin frontend (React + Tailwind) and Django + DRF backend with authentication
- Institutional knowledge base ingestion pipeline (PyMuPDF → embeddings → pgvector)
- AI Campus Query Bot answering questions grounded in retrieved context (RAG with GPT-4o-mini)
- 2D campus map for locating classrooms, departments, and event venues
- Student verification via institutional email check
- CI: build + backend unit tests; CD to staging (Render/Railway, Vercel)

10.2 Subsequent deliverables

- Interactive 3D campus navigation (Three.js + Blender models) with turn-by-turn direction

- Personalized dashboard and reminders for academic/campus dates
- Admin dashboard for knowledge-base maintenance across multiple colleges
- Voice assistant, mobile app, university ERP and attendance integration, personalized academic assistant

11 Risks and mitigations

- **Ungrounded/incorrect answers:** rely on RAG evaluation with a test dataset before release; never let the LLM answer without retrieved context for institution-specific questions.
- **Student verification bypass:** keep verification (institutional email check) mandatory before institution-specific data is served.
- **Knowledge-base staleness:** give admins a straightforward Django Admin workflow to update documents so the vector store reflects current information.
- **Scaling to multiple colleges:** keep institution data and knowledge bases logically separated from the start so onboarding a new college does not require re-architecture.

12 Summary

This project proposes Athena, a deployable, multi-college AI campus assistant that centralizes fragmented campus information behind a single natural-language interface. It meets the course criteria by emphasizing time-to-value through rapid iterations starting with a single-college pilot, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria centered on grounded, relevant answers. Its contribution lies in system-level integration — verified institutional knowledge, student verification, semantic search, retrieval-augmented generation, and an interactive campus map — rather than research-driven novelty in any single component.